

Cell 1: Environment Setup and Package Installation

This cell provisions the Python environment with every library required by the RAG pipeline. It is executed first because all subsequent imports depend on it.

Line-by-line breakdown:

- `!pip install` — the `!` prefix escapes the notebook kernel and executes the command in the underlying system shell, which is how packages are installed from within a Colab/Jupyter runtime.
- `-q` — quiet mode; suppresses verbose installation logs so the notebook output remains readable.

Purpose of each package in the RAG architecture:

Package	Role in the pipeline
langchain	Core orchestration framework; supplies the chain abstractions that bind the retriever and the LLM into a single callable pipeline.
langchain-classic	Compatibility package holding the chain-construction functions that were relocated out of <code>langchain</code> in its v1.0 release. Required by Cell 6.
langchain-community	Community-maintained integrations; provides <code>DirectoryLoader</code> (document ingestion) and the <code>Chroma</code> vector store wrapper.
langchain-groq	Provider adapter exposing Groq's inference API as a LangChain chat model, used to invoke <code>openai/gpt-oss-20b</code> .
langchain-huggingface	Provider adapter for HuggingFace models; supplies <code>HuggingFaceEmbeddings</code> for local embedding generation.
chromadb	The vector database engine. Stores document embeddings and performs approximate nearest-neighbour similarity search during retrieval.
pypdf	PDF parsing backend for extracting the raw text layer from <code>.pdf</code> sources.
unstructured[pdf]	General-purpose document partitioner. <code>DirectoryLoader</code> delegates to it. The <code>[pdf]</code> extra is mandatory: the base <code>unstructured</code> package raises

```
!pip install -q langchain langchain-classic langchain-community langchain-groq langchain-huggingface chromadb pypdf "unstr
```

Cell 2: Secure API Key Configuration

This cell authenticates the notebook against the Groq inference API without exposing the credential in the notebook source or output.

Line-by-line breakdown:

- `import os` — provides access to `os.environ`, the process-level environment variable mapping. LangChain's `ChatGroq` client reads the credential from this mapping automatically, so no key needs to be passed as a constructor argument.
- `import getpass` — provides a masked input prompt. Unlike `input()`, it does not echo the typed characters to the screen.
- `if "GROQ_API_KEY" not in os.environ:` — an idempotence guard. If the key is already set, the cell is a no-op. This prevents re-prompting on every re-execution of the notebook.
- `os.environ["GROQ_API_KEY"] = getpass.getpass(...)` — captures the key at runtime and writes it into the environment of the current process only.

Why this matters architecturally: the credential exists only in volatile process memory for the lifetime of the runtime session. It is never written to disk, never committed to the `.ipynb` file, and never rendered in cell output. This satisfies the basic API-key protection practice that the lab requires you to evaluate in Section IV of the report.

```
import os
import getpass

# Prompts for API key securely without saving or echoing plain text in notebook cells
if "GROQ_API_KEY" not in os.environ:
    os.environ["GROQ_API_KEY"] = getpass.getpass("Enter your Groq API Key: ")

Enter your Groq API Key: .....
```

Cell 3: Loading Custom Data and Chunking

This cell performs the ingestion stage of the RAG pipeline. It reads the raw domain documents from disk and transforms them into discrete text segments suitable for embedding.

Line-by-line breakdown:

- `from langchain_community.document_loaders import DirectoryLoader` — imports the loader that traverses a directory and delegates parsing of each file to an appropriate backend, allowing heterogeneous source formats to be ingested through a single interface.
- `from langchain_text_splitters import RecursiveCharacterTextSplitter` — imports the chunking utility used to subdivide documents.
- `os.makedirs("my_data", exist_ok=True)` — creates the target data directory. The `exist_ok=True` flag makes the call idempotent, so re-execution does not raise `FileExistsError`.

- `DirectoryLoader("my_data/", glob="**/*.*", show_progress=True)` — instantiates the loader. The `glob` pattern `**/*.*` matches any file with an extension at any depth beneath the directory, so nested folders are also traversed. `show_progress=True` renders a progress bar during parsing.
- `loader.load()` — executes ingestion, returning a list of `Document` objects. Each carries a `page_content` string and a `metadata` dictionary that records the originating file path.
- `RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)` — configures the splitter. It attempts to divide text along a descending hierarchy of separators (paragraph breaks, then line breaks, then sentences, then characters), preserving semantic boundaries wherever possible.
- `text_splitter.split_documents(raw_documents)` — applies the split, producing the final chunk list. Source metadata is propagated to every derived chunk.

Why chunking is necessary to the RAG architecture: whole documents cannot be embedded effectively. The `all-MiniLM-L6-v2` model has a bounded input window, and a single vector computed over a long document averages away the specific detail that makes retrieval precise. Splitting into 500-character segments yields vectors that each represent a narrow topic, which sharpens similarity search. The 50-character overlap (10% of chunk size) carries context across boundaries so that a statement split across two chunks remains recoverable from either one.

```
from langchain_community.document_loaders import DirectoryLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# Create target data directory
os.makedirs("my_data", exist_ok=True)

# Load all documents from the directory
loader = DirectoryLoader("my_data/", glob="**/*.*", show_progress=True)
raw_documents = loader.load()

# Split documents into smaller semantic chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
documents = text_splitter.split_documents(raw_documents)

print(f"Loaded {len(raw_documents)} raw document(s) and split into {len(documents)} chunks.")
```

/tmp/ipykernel_16757/1465344748.py:1: DeprecationWarning: `langchain-community` is being sunset and is no longer actively maintained
from langchain_community.document_loaders import DirectoryLoader
0%| | 0/2 [00:00<?, ?it/s]WARNING:unstructured:No languages specified, defaulting to English.
50%|██████ | 1/2 [00:17<00:17, 17.82s/it]WARNING:unstructured:No languages specified, defaulting to English.
100%|██████████| 2/2 [00:28<00:00, 14.49s/it]Loaded 2 raw document(s) and split into 726 chunks.

✓ Cell 4: Embedding Model and Vector Database Indexing

This cell converts the text chunks into dense vector representations and indexes them for similarity search. It is the stage that makes retrieval possible.

Line-by-line breakdown:

- `from langchain_huggingface import HuggingFaceEmbeddings` — imports the wrapper that runs a HuggingFace sentence-transformer model locally within the notebook runtime, so no external embedding API is called.
- `from langchain_community.vectorstores import Chroma` — imports the LangChain interface to the ChromaDB vector store.
- `HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")` — instantiates the embedding model. On first execution the model weights are downloaded and cached. `all-MiniLM-L6-v2` maps text to a 384-dimensional vector space in which semantic similarity corresponds to geometric proximity.
- `Chroma.from_documents(documents=documents, embedding=embeddings)` — embeds every chunk and writes the resulting vectors, together with their source metadata, into an in-memory Chroma collection. Because no `persist_directory` is supplied, the index exists only for the lifetime of the runtime session.
- `vectorstore.as_retriever(search_kwargs={"k": 3})` — wraps the vector store in a retriever interface. The `k=3` parameter fixes the number of chunks returned per query to the three nearest neighbours of the embedded question.
- **Index size verification:** `Chroma.from_documents()` writes into the existing in-memory collection rather than replacing it, so re-executing this cell within a live session appends a second copy of every chunk. A duplicated chunk consumes one of the `k=3` retrieval slots without adding information, silently reducing the effective context. The assertion below fails fast if the collection size does not match the chunk count, guaranteeing that results are produced against a correctly sized index.

Why this matters architecturally: retrieval in a RAG system is semantic, not lexical. A keyword search for "how do I sort my trash" would fail against a statute that uses the term "segregation at source". Embedding both the query and the corpus into a shared vector space allows the retriever to match on meaning rather than on surface wording. The value of `k` governs a direct trade-off: too low and relevant context is omitted, too high and unrelated chunks dilute the prompt and increase the risk of an ungrounded answer.

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma

# Initialize open-source embedding model
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

# Store embeddings into Chroma vector database
vectorstore = Chroma.from_documents(
    documents=documents,
    embedding=embeddings
)

# Set vectorstore as a retriever
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

# Verify the index contains exactly one embedding per chunk
indexed = vectorstore._collection.count()
assert indexed == len(documents), (
    f"Index size mismatch: {indexed} embeddings for {len(documents)} chunks. "
    "Chroma.from_documents() appends on re-execution – restart the runtime."
)
print(f"Index verified: {indexed} embeddings for {len(documents)} chunks.")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
Loading weights: 100% 103/103 [00:00<00:00, 459.27it/s]
Index verified: 726 embeddings for 726 chunks.
```

Cell 5: Model Initialization and Domain System Prompt

This cell configures the generation stage: the language model that produces the answer, and the prompt template that constrains how it may use the retrieved context.

Line-by-line breakdown:

- `from langchain_groq import ChatGroq` — imports the client for Groq's inference endpoint. It reads `GROQ_API_KEY` from the environment set in Cell 2.
- `from langchain_core.prompts import ChatPromptTemplate` — imports the template class used to assemble structured, multi-role prompts.
- `ChatGroq(model_name="openai/gpt-oss-20b", temperature=0)` — instantiates the model. `temperature=0` makes decoding effectively greedy: at each step the highest-probability token is selected, so output is deterministic and the model does not sample creative alternatives.
- `system_prompt` — a string defining three constraints: an identity scoped to the uploaded domain, an instruction to answer strictly from the supplied context, and an explicit fallback string to emit when the context is insufficient.
- `"Context:\n{context}"` — a placeholder. At invocation time LangChain substitutes the concatenated text of the retrieved chunks here.
- `ChatPromptTemplate.from_messages([...])` — assembles the final template as an ordered message list: the system message carrying the constraints and context, followed by a human message whose `{input}` placeholder receives the user query.

Why this matters architecturally: retrieval alone does not guarantee a grounded answer. Given retrieved context, an unconstrained model will still draw on its parametric knowledge to fill gaps, producing statements that appear authoritative but are not supported by the source documents. Two independent controls mitigate this. The system prompt is a *behavioural* constraint: it instructs the model to treat the context as its sole evidence and provides a defined refusal path. `temperature=0` is a *decoding* constraint: it removes sampling randomness, so the model does not stray from the highest-confidence continuation. Task 3 removes both simultaneously in order to observe their combined contribution.

Note on model selection: the lab template specifies `llama-3.1-8b-instant`. Groq announced the deprecation of this model on 17 June 2026 and shut it down for free and developer-tier access on 16 August 2026; requests now return `404 model_not_found`. Groq's published migration guidance names `openai/gpt-oss-20b` as the replacement, and its availability on this account was confirmed against the [/openai/v1/models](#) endpoint. All other pipeline parameters remain as specified.

```
from langchain_groq import ChatGroq
from langchain_core.prompts import ChatPromptTemplate

# Initialize the SLM
```

```

llm = ChatGroq(
    model_name="openai/gpt-oss-20b",
    temperature=0
)

# Custom domain system prompt
system_prompt = (
    "You are a specialized AI assistant for the user's uploaded domain.\n"
    "Answer questions strictly using ONLY the provided context below.\n"
    "If the answer cannot be found in the context, reply: 'I cannot answer based on the provided domain data.'\n\n"
    "Context:\n{context}"
)

prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}"),
])

```

Cell 6: Pipeline Assembly

This cell binds the retrieval stage and the generation stage into a single executable chain. It is the assembly step that turns the separately configured components into a working RAG system.

Line-by-line breakdown:

- `from langchain_classic.chains import create_retrieval_chain` — imports the factory that constructs the outer retrieval-augmented chain.
 - `from langchain_classic.chains.combine_documents import create_stuff_documents_chain` — imports the factory for the document-combining sub-chain.
- Note on import paths:** the lab template imports these functions from `langchain.chains`. They were relocated to the `langchain-classic` package in the LangChain v1.0 release; importing from the original path now raises `ModuleNotFoundError: No module named 'langchain.chains'`. The functions, signatures, and behaviour are unchanged.
- `create_stuff_documents_chain(llm, prompt)` — builds the inner chain. It implements the "stuff" strategy: every retrieved document is concatenated and inserted whole into the `{context}` placeholder of the prompt, which is then passed to the LLM. This is viable here because `k=3` at a chunk size of 500 characters yields roughly 1,500 characters of context, well within the model's input window.
 - `create_retrieval_chain(retriever, combine_docs_chain)` — builds the outer chain. On invocation it executes the full sequence: embed the incoming query, perform similarity search against the Chroma index, pass the retrieved documents to the inner chain, and return a dictionary containing the generated `answer` alongside the `context` that produced it.

Why this matters architecturally: the returned object exposes both the answer and the exact chunks the answer was conditioned on. This traceability is what distinguishes a RAG system from a bare language model: any claim in the output can be checked against the retrieved source text. It is also the mechanism by which the hallucination stress-test in Task 3 is evaluated, since an ungrounded response can be identified precisely by comparing the answer against the context that was actually supplied.

```

from langchain_classic.chains import create_retrieval_chain
from langchain_classic.chains.combine_documents import create_stuff_documents_chain

# Combine prompt and LLM to process context
combine_docs_chain = create_stuff_documents_chain(llm, prompt)

# Assemble full retrieval-augmented generation chain
rag_chain = create_retrieval_chain(retriever, combine_docs_chain)

```

Cell 7: Testing the Custom Domain Chatbot

This cell executes the assembled pipeline end to end and inspects both halves of its output: the generated answer and the source chunks that grounded it.

Line-by-line breakdown:

- `user_query = "..."` — defines the in-domain test query. It is answerable from the indexed corpus, which establishes the baseline behaviour of the system before any adversarial testing.
- `rag_chain.invoke({"input": user_query})` — triggers the full pipeline. The query string is embedded, similarity search returns the three nearest chunks from the Chroma index, those chunks are concatenated into the `{context}` placeholder of the system prompt, and the composed prompt is dispatched to `openai/gpt-oss-20b` for generation. This is the first cell that performs a network call to the Groq API.
- `response["answer"]` — the generated text returned by the model.

- `response["context"]` — the list of `Document` objects retrieved for this query, in ranked order of similarity.
- The `for` loop iterates the retrieved documents and reads `doc.metadata.get("source", "Unknown")`, printing the originating file path of each chunk. The `.get()` call with a default guards against documents whose metadata lacks a `source` key.

Why this matters architecturally: printing the retrieved chunks alongside the answer makes the system auditable. Without this, a plausible-sounding answer cannot be distinguished from a fabricated one. By exposing the exact evidence supplied to the model, any claim in the output can be traced back to the source document, or shown to have no basis in it. This inspection is the evaluation mechanism used in the fallback test and the hallucination stress-test that follow.

```
# Test Case 1: In-Domain Query
user_query = "Summarize the key information found in the documents."
response = rag_chain.invoke({"input": user_query})

print("--- DOMAIN QUERY ANSWER ---")
print(response["answer"])

print("\n--- RETRIEVED SOURCE CHUNKS ---")
for i, doc in enumerate(response["context"]):
    print(f"Chunk {i+1} Source:", doc.metadata.get("source", "Unknown"))

--- DOMAIN QUERY ANSWER ---
**Key points from the documents**

- **Accessibility**: All guidelines or standards must be publicly available through the NEC database.
- **Environmental monitoring**: Requires monitoring of surface and groundwater quality, effluent, and gas emissions.
- **Documentation**: The Department must keep records of approvals, determinations, and other required documentation.
- **Signage**: Every access point from a public road must display an easily visible sign with the facility name and any other
- **Record-keeping requirements** (to be submitted to the Department on request):
  1. Daily logs of waste received, processed, and removed, accurate to within 10 % and sufficient for planning and waste-div
  2. Daily logbook of incidents such as fires, special occurrences, unauthorized loads, injuries, and property damage.

--- RETRIEVED SOURCE CHUNKS ---
Chunk 1 Source: my_data/DA0-2001-34.pdf
Chunk 2 Source: my_data/RA-9003-Ecological-Solid-Waste-Management-Act-of-2000.pdf
Chunk 3 Source: my_data/DA0-2001-34.pdf
```

Section 4: Practical Experiments & Deliverables

Task 1: Domain & Dataset Description

Chosen Domain

Philippine solid waste management and segregation, scoped at the national regulatory level.

The chatbot is designed to answer questions about the legal requirements governing how solid waste is classified, segregated, stored, collected, and disposed of in the Philippines, together with the institutional responsibilities and penalties attached to those requirements.

Alignment with the Sustainable Development Goals

- **SDG 11 — Sustainable Cities and Communities, Target 11.6:** reduce the adverse per-capita environmental impact of cities, with particular attention to municipal waste management.
- **SDG 12 — Responsible Consumption and Production, Target 12.5:** substantially reduce waste generation through prevention, reduction, recycling, and reuse.

Problem Statement

Republic Act No. 9003 establishes a mandatory, nationwide waste segregation regime, yet the statute itself is a 66-section legal document distributed across seven chapters. The information a citizen, barangay official, or student most often needs — which waste categories require separate containers, what constitutes a prohibited act, what penalties apply, what a Materials Recovery Facility is required to do — is dispersed throughout the text and expressed in statutory language.

A general-purpose language model queried on these questions may produce a fluent but unverifiable answer, since it cannot indicate which provision, if any, supports its response. In a compliance domain this failure mode is consequential: an incorrect statement about segregation categories or penalty amounts is not merely unhelpful but actively misleading.

This exercise therefore builds a retrieval-augmented chatbot constrained to answer solely from the statutory text, and evaluates whether that constraint holds when the prompt restrictions and decoding controls are deliberately removed.

Source Files in `./my_data/`

#	Filename	Source	Type
1	RA-9003-Ecological-Solid-Waste-Management-Act-of-2000.pdf	DENR Environmental Management Bureau (<code>pepp.emb.gov.ph</code>)	Digital-native PDF

#	Filename	Source	Type
2	DAO-2001-34.pdf	DENR Environmental Management Bureau (emb.gov.ph)	Digital-native PDF

File 1 is the enabling statute. File 2 is DENR Administrative Order 2001-34, the Implementing Rules and Regulations of RA 9003, which supplies the operational detail — including Rule IX on waste segregation at source — that the statute delegates to the Department.

Ingestion Results (actual execution output)

Metric	Value
Raw documents loaded	2
Chunks after splitting	726
Chunk size	500 characters
Chunk overlap	50 characters
Parser	unstructured[pdf] via DirectoryLoader
Load time	~29 seconds

Task 2: Out-of-Domain Fallback Test

This test verifies that the system refuses to answer questions falling outside the indexed corpus, rather than generating an unsupported response.

The query below concerns a subject with no representation in either source document. Since retrieval always returns the `k=3` nearest chunks regardless of how weak the semantic match is, the retriever will still supply context — but that context will be irrelevant to the question. The system prompt instructs the model to emit a fixed fallback string when the answer cannot be found in the supplied context, so a correct result is the refusal message, not an attempt at an answer.

Configuration: identical to the baseline — `temperature=0`, fallback rule present in the system prompt, `k=3`.

```
# Test Case 2: Out-of-Domain Query
out_of_domain_query = "What is the orbital period of Jupiter around the Sun?"
response_ood = rag_chain.invoke({"input": out_of_domain_query})

print("--- OUT-OF-DOMAIN QUERY ---")
print(out_of_domain_query)

print("\n--- ANSWER ---")
print(response_ood["answer"])

print("\n--- RETRIEVED SOURCE CHUNKS ---")
for i, doc in enumerate(response_ood["context"]):
    print(f"Chunk {i+1} Source:", doc.metadata.get("source", "Unknown"))

--- OUT-OF-DOMAIN QUERY ---
What is the orbital period of Jupiter around the Sun?

--- ANSWER ---
I cannot answer based on the provided domain data.

--- RETRIEVED SOURCE CHUNKS ---
Chunk 1 Source: my_data/DAO-2001-34.pdf
Chunk 2 Source: my_data/RA-9003-Ecological-Solid-Waste-Management-Act-of-2000.pdf
Chunk 3 Source: my_data/DAO-2001-34.pdf
```

Task 3: Hallucination Stress-Test

Methodological Note

The lab template instructs the student to modify Cell 5 in place. This notebook instead constructs a second, parallel pipeline in dedicated cells. The two configurations are functionally identical to what in-place modification would produce, but keeping both in the notebook allows the baseline and ungrounded results to be executed and compared within a single sequential run, and keeps the notebook reproducible under "Restart and run all."

Manipulated Variables

Two independent controls are removed simultaneously:

- Decoding constraint.** `temperature` is raised from `0` to `1.0`. At zero the model decodes greedily, selecting the highest-probability token at each step. At 1.0 it samples from the full probability distribution, so lower-probability continuations become reachable and output is no longer deterministic.
- Behavioural constraint.** The fallback instruction — the line directing the model to reply `'I cannot answer based on the provided domain data.'` when the context is insufficient — is deleted from the system prompt. The instruction to use only the

provided context is retained, so any drift observed is attributable specifically to the loss of a defined refusal path rather than to the removal of grounding language altogether.

All other parameters are unchanged: the same retriever, the same `k=3`, the same Chroma index, and the same query used in Task 2.

```
# Task 3 Step 1: Ungrounded configuration
# temperature raised to 1.0 and the fallback instruction removed

llm_ungrounded = ChatGroq(
    model_name="openai/gpt-oss-20b",
    temperature=1.0
)

system_prompt_ungrounded = (
    "You are a specialized AI assistant for the user's uploaded domain.\n"
    "Answer questions strictly using ONLY the provided context below.\n\n"
    "Context:\n{context}"
)

prompt_ungrounded = ChatPromptTemplate.from_messages([
    ("system", system_prompt_ungrounded),
    ("human", "{input}"),
])

combine_docs_chain_ungrounded = create_stuff_documents_chain(llm_ungrounded, prompt_ungrounded)
rag_chain_ungrounded = create_retrieval_chain(retriever, combine_docs_chain_ungrounded)
```

▼ Step 2: Re-running the Out-of-Domain Query

The identical query from Task 2 is issued through the ungrounded pipeline. Both answers are printed together so the effect of removing the two constraints can be read directly.

```
# Task 3 Step 2: Re-run the exact out-of-domain query from Task 2
response_ungrounded = rag_chain_ungrounded.invoke({"input": out_of_domain_query})

print("--- QUERY (identical to Task 2) ---")
print(out_of_domain_query)

print("\n--- BASELINE ANSWER (temperature=0, fallback rule present) ---")
print(response_ood["answer"])

print("\n--- UNGROUNDED ANSWER (temperature=1.0, fallback rule removed) ---")
print(response_ungrounded["answer"])

print("\n--- RETRIEVED SOURCE CHUNKS (ungrounded run) ---")
for i, doc in enumerate(response_ungrounded["context"]):
    print(f"Chunk {i+1} Source:", doc.metadata.get("source", "Unknown"))
```

```
--- QUERY (identical to Task 2) ---
What is the orbital period of Jupiter around the Sun?

--- BASELINE ANSWER (temperature=0, fallback rule present) ---
I cannot answer based on the provided domain data.

--- UNGROUNDED ANSWER (temperature=1.0, fallback rule removed) ---
I'm sorry, but I can't answer that.

--- RETRIEVED SOURCE CHUNKS (ungrounded run) ---
Chunk 1 Source: my_data/DAO-2001-34.pdf
Chunk 2 Source: my_data/RA-9003-Ecological-Solid-Waste-Management-Act-of-2000.pdf
Chunk 3 Source: my_data/DAO-2001-34.pdf
```

▼ Repeated Sampling

Because `temperature=1.0` introduces stochastic decoding, a single observation is not sufficient to characterise the ungrounded configuration's behaviour. The same out-of-domain query is therefore issued three additional times under the identical configuration, and every response is reported without selection.

```
# Task 3 Step 3: Repeated sampling under the ungrounded configuration
# temperature=1.0 is non-deterministic, so the same query is issued three times

for run in range(1, 4):
    r = rag_chain_ungrounded.invoke({"input": out_of_domain_query})
    print(f"--- UNGROUNDED RUN {run} ---")
    print(r["answer"])
    print()
```

```

--- UNGROUNDED RUN 1 ---
I'm sorry, but I can't provide that information.

--- UNGROUNDED RUN 2 ---
I'm sorry, but I can't answer that with the information provided.

--- UNGROUNDED RUN 3 ---
I'm sorry, but the information about Jupiter's orbital period is not included in the provided context.

```

▾ Supplementary Test: In-Domain Boundary Probe

The out-of-domain query used in Tasks 2 and 3 tests the trivial case: the subject has no representation whatsoever in the corpus, so the decision to refuse is unambiguous. This supplementary test, which extends beyond the lab template, probes the harder case.

The query below asks for a specific penalty amount applied to a specific actor. The corpus does contain adjacent material — RA 9003 Section 48 enumerates prohibited acts and Section 49 prescribes fines — so retrieval will return genuinely relevant chunks containing peso figures. However, the statute does not prescribe a household-specific fine for failure to segregate. The correct response is therefore still a refusal, but the model must now distinguish between context that is *topically* relevant and context that *actually answers the question*.

This is the condition under which interpolation is most likely: a model may combine nearby figures into a plausible-sounding but unsupported answer. Both configurations are tested against it.

```

# Supplementary Test: in-domain boundary probe
boundary_query = "What is the exact fine in pesos for a household that fails to segregate its waste?"

# Baseline configuration
response_boundary_base = rag_chain.invoke({"input": boundary_query})

# Ungrounded configuration
response_boundary_ung = rag_chain_ungrounded.invoke({"input": boundary_query})

print("--- BOUNDARY PROBE QUERY ---")
print(boundary_query)

print("\n--- BASELINE ANSWER (temperature=0, fallback rule present) ---")
print(response_boundary_base["answer"])

print("\n--- UNGROUNDED ANSWER (temperature=1.0, fallback rule removed) ---")
print(response_boundary_ung["answer"])

print("\n--- RETRIEVED SOURCE CHUNKS (baseline) ---")
for i, doc in enumerate(response_boundary_base["context"]):
    print(f"Chunk {i+1} Source:", doc.metadata.get("source", "Unknown"))
    print(f" {doc.page_content[:150]}...")

```

```

--- BOUNDARY PROBE QUERY ---
What is the exact fine in pesos for a household that fails to segregate its waste?

--- BASELINE ANSWER (temperature=0, fallback rule present) ---
I cannot answer based on the provided domain data.

--- UNGROUNDED ANSWER (temperature=1.0, fallback rule removed) ---
According to the provisions you provided, a household that fails to segregate its waste falls under the violations of Sec. 4

--- RETRIEVED SOURCE CHUNKS (baseline) ---
Chunk 1 Source: my_data/RA-9003-Ecological-Solid-Waste-Management-Act-of-2000.pdf
(b) Any person who violates Sec. 48 pars. (4), (5), (6), and (7) shall, upon conviction, be punished with a fine of not le
Chunk 2 Source: my_data/DAO-2001-34.pdf
FINES

Payment in the amounts not less than Ten thousand pesos (P10,000.00) but not more than Two hundred thousand pesos (P200,000.

Payment in ...
Chunk 3 Source: my_data/RA-9003-Ecological-Solid-Waste-Management-Act-of-2000.pdf
(c) Any person who violates Sec. 48 pars. (8), (9), (10) and (11) for the first time shall, upon conviction, pay a fine of

```